Use case: AL-11 Messaging

Epic. A borrower messages their broker or the bank's lending officer inside the app, and the broker and the underwriter answer from the portal, in one thread per application. This is a new Epic in Sprint 2, so this is its first use case.

What changed in Sprint 2

- **The backend is built (AL-70, AL-71, Dumi, Done).** Messages live under each application. Clients only read them; every message is sent through al-core, which sets the sender from the sign-in token. Each person has their own read position, so the borrower, the broker and each underwriter get their own unread count.
- **The borrower's side is built (AL-42, Ibrahima, Done).** The iOS app shows a live thread with the broker, sends messages through al-core, marks replies read and offers "try again" on a failed send.
- **The broker's inbox is in review (AL-43, PR #193, approved, not merged).** One thread per client application, unread first, with a reply box.
- **The underwriter's reply is in review (AL-152, PR #194, approved, not merged).** "Message borrower" on the loan file opens the same thread; the message is labelled "Bank lending officer".

Overview

Item	Detail
Epic	AL-11 Messaging
Feature (from the brief)	Mobile: Broker / Official Contact: in-app messaging to communicate directly with the assigned mortgage broker or bank lending officer.
Owner	Nikoloz Chilachava (portal: broker inbox and underwriter reply). The borrower's thread in the iOS app is Ibrahima's (AL-42). The messages model, endpoints and read state are Dumi's (AL-70, AL-71).
Sprint	Sprint 2 (27 Oct to 15 Nov 2026).
Wireframes	Portal: Figma V2 frames "Broker / Messages Inbox", "Broker / Message Drawer" and "Underwriter / Message Drawer" (file `F6lldwqy172ox5SeALdDEU`). Borrower app: Figma messages frames 15 to 15e.

Goal

As a borrower, I want to message my broker or lending officer inside the app, so that questions about my application don't get lost in email.

Actors

- **Borrower:** signed in to the iOS app, with a submitted application. Reads and sends in the thread for that application.
- **Broker:** signed in to the portal with the broker role. Reads and replies only in the threads of their own clients' applications.
- **Underwriter (bank lending officer):** signed in to the portal with the underwriter role. Can open the thread of any submitted application from its loan file and reply as "Bank lending officer".
- **al-core API:** the only writer. Sends a message, marks messages read and gives the unread count. It checks the sign-in token and who is in the thread.
- **Firestore:** stores the thread and each person's read position. The app and the portal read them straight from Firestore and listen live; the security rules let only the thread's people read them.

Preconditions

- The application has been submitted (status Applied or later). A Draft has no thread, because a broker can't see a Draft.
- The application has a `borrowerId` and a `brokerId`.
- The person is signed in (Firebase Auth) and their ID token carries the `role` claim (`borrower`, `broker` or `underwriter`).
- al-core is running and reachable; for local work it runs against the Firebase emulators.

Trigger: the borrower opens Messages in the app and sends a question about their application.

Main flow

1. The borrower opens Messages. The app listens to `applications/{id}/messages` for their application and shows one thread with their broker.
2. The borrower types a message and taps Send. The app sends it with `POST /core/applications/{id}/messages`, and shows it straight away while it sends.
3. al-core checks that the caller is in the thread, then saves the message with `senderId` and `senderRole` from the token, the text, `sentAt` set by the server, and `read` false.
4. The broker opens Messages in the portal (`/broker/messages`). The inbox lists one thread per client application, with each thread's last message and the broker's unread count, unread threads first. The list updates live.
5. The broker opens the borrower's thread. The portal shows it beside the list and calls `POST /core/applications/{id}/messages/read`, which marks the borrower's messages read and moves the broker's own `readState` to the newest message.
6. The broker types a reply and sends it through the same endpoint. The reply box clears only once al-core has saved it.
7. The borrower's app, listening to the thread, shows the reply with an unread badge until the thread is opened. Opening it marks the reply read for the borrower.

Alternative flows

- **A. The underwriter answers as the bank (AL-152).** At step 4, an underwriter opens the application's loan file and clicks "Message borrower". The same thread opens beside the loan file. The underwriter's message is saved with `senderRole` underwriter, and the portal shows it to everyone else as "Bank lending officer" and to the sender as "You".
- **B. Nothing new.** At step 5, if nothing is unread, the portal does not call the read endpoint.
- **C. A message arrives while the thread is open.** It shows at once in the open thread. The person's read position stays at the newest message they had when they opened it, so a message that arrives during the open still counts as unread for them.
- **D. A thread with no messages.** The inbox row and the thread say "No messages yet".

Exceptions

- **Empty or blank message:** the portal keeps Send disabled; al-core also refuses an empty text, or one over 2,000 characters, with a validation error.
- **Someone outside the thread:** a broker sending or reading on another broker's client's application, or a borrower on someone else's, gets the same 404 as an unknown application id from al-core, and the Firestore rules refuse the read. Nobody can probe which ids exist.
- **Draft application:** al-core answers 409 "Messages open once the application is submitted"; the portal shows no "Message borrower" button on a Draft or to a broker on the loan file.
- **Send fails (no connection, al-core down):** the portal keeps the typed text and says "Could not reach the server. Check your connection and try again." (or "Could not send the message. Please try again." for other errors). The iOS app marks the message "Not sent. Tap to try again" and tapping sends it again without a duplicate.
- **The thread can't be read:** the portal says "The messages could not be loaded. Reload the page to try again." instead of showing an empty thread, and stops listening when the thread is closed.

Postconditions

- **Success:** the message is stored once at `applications/{id}/messages/{messageId}` with a trusted sender and server time, every person in the thread sees it live, and each person's unread count reflects what they have opened.
- **Failure:** nothing is stored, and the sender can try again.

Business rules

- One thread per application, shared by the borrower, the assigned broker and the underwriters. There is no separate web-only thread.
- Only al-core writes messages and read state. The sender's id and role come from the token, never from the request body, so the portal labels "Bank lending officer" from stored data, not from what the sender typed.

- A sent message cannot be edited or deleted: there is no endpoint for either.
- Two kinds of "read": the `read` flag on a message means the other side has read it (borrower versus bank staff), and is what the borrower's app shows. `applications/{id}/readState/{uid}` holds each person's own `lastReadAt`, and the portal works out the broker's and the underwriter's unread counts from it: messages sent by someone else after `lastReadAt`, or all of them before the first open.
- A broker opening the thread never clears the underwriter's message to the borrower, and the other way round.
- When an application moves to another broker, the old broker loses access because `brokerId` changes.
- Text only. Attachments would need Firebase Storage, and push notifications are a stretch goal.

Data

Path	Fields	Who reads	Who writes
`applications/{id}/messages/{messageId}`	`senderId`, `senderRole` (`borrower`, `broker` or `underwriter`), `text`, `sentAt`, `read`	The borrower, the assigned broker, underwriters	al-core only
`applications/{id}/readState/{uid}`	`lastReadAt`, `role`	That person only	al-core only

Endpoint	What it does
`POST /core/applications/{id}/messages`	Sends a message as the caller
`POST /core/applications/{id}/messages/read`	Marks the caller's side's unread messages read and moves the caller's `lastReadAt`
`GET /core/applications/{id}/messages/unread`	The caller's unread count and last read time

Stories in this epic

Story	What	Owner	Sprint	Status on 12 Nov
AL-42	As a borrower, I message my assigned broker or lending officer in the app	Ibrahima	Sprint 2	Done: live thread (#132), send and mark read through al-core (#151)
AL-43	As a broker, I reply to borrower messages from the portal	Nikoloz	Sprint 2	In Review: PR #193 open, approved

AL-152	As an underwriter, I reply to a borrower's messages as the bank lending officer	Nikoloz	Sprint 2	In Review: PR #194 open, approved

Backend subtasks: AL-70 messages model and endpoints (Dumi, PR #150, merged 30 Oct) and AL-71 per-person read state (Dumi, PR #164, merged 5 Nov). Both Done.

Not built yet, and known limits

- The portal side (AL-43, AL-152) is not on `main` until PRs #193 and #194 merge.
- The Epic's "done when" check, a message from the seeded borrower showing in the broker's inbox and the reply showing in the app with no refresh on either side, has not been run end to end. The borrower's side was checked by Ibrahima (AL-42 test case); the portal side was checked against seeded messages and the underwriter's reply (AL-43 and AL-152 test cases).
- On the local emulator, the broker inbox previews once stayed on "Loading..." until other listeners started; Dumi could not reproduce it (12 Nov). It is noted in the AL-43 test case.
- Whether the iOS app labels an underwriter's message with its role has not been checked in the app.
- No push notifications and no attachments.

Notes

- Test cases: `AL-43/test-case.pdf` and `AL-152/test-case.pdf` in this folder; `ibrahima/AL-42/test-case.pdf` for the borrower's side.
- Field details: `al-core/SCHEMA.md` on `main`, sections for `applications/{id}/messages/{messageId}` and `applications/{id}/readState/{uid}`.